Prompt-mestre para geração do sistema Core do Smart Campus

Instrução de utilização

Copie integralmente o conteúdo a partir da secção **Prompt a entregar à IA** e forneça-o à IA de desenvolvimento. Se a IA precisar de escolher tecnologias, deverá adoptar as decisões técnicas indicadas neste documento e justificar qualquer alteração antes de implementar.

Prompt a entregar à IA

1. Papel que deves assumir

Actua como **arquitecto de software, engenheiro full-stack, especialista em APIs, segurança, bases de dados, Git, testes, integração modular e desenvolvimento pedagógico baseado em projectos**.

Deves gerar uma implementação funcional, organizada, documentada e preparada para integração por vários grupos de estudantes. Não deves criar uma aplicação isolada nem um protótipo descartável. Deves criar a base de uma **única plataforma integrada**, denominada **Smart Campus**, que será desenvolvida progressivamente no âmbito da unidade curricular **Prática Técnico-Profissional III — PTP III**, da Universidade Joaquim Alberto Chissano — UJAC.

A solução deve ser suficientemente simples para ser compreendida e evoluída por estudantes, mas deve aplicar práticas profissionais de arquitectura, segurança, versionamento, testes, documentação e integração.

2. Objectivo principal da cadeira

O objectivo principal não é produzir seis aplicações independentes. O objectivo é que a turma construa, colaborativamente, **uma única plataforma Smart Campus**, composta por um Core comum e por módulos integrados.

> Seis grupos não devem produzir seis aplicações isoladas; devem construir seis módulos de uma única plataforma integrada.

O sistema deve permitir que os grupos trabalhem em paralelo sem duplicar autenticação, utilizadores, base de dados, convenções de API, regras de segurança, configuração ou

componentes de interface.

A solução inicial deve criar a fundação comum e apresentar no frontend todos os módulos previstos, mesmo que alguns ainda estejam em desenvolvimento. Os módulos que ainda não possuírem funcionalidade completa devem aparecer como áreas preparadas, claramente identificadas como “Em desenvolvimento”, sem quebrar a navegação nem fingir que já estão implementados.

3. Produto a gerar

Gera um repositório funcional chamado `smart-campus-core` ou `smart-campus`, contendo:

1. Backend Core modular.
2. API REST versionada.
3. Base de dados relacional.
4. Autenticação e autorização.
5. Utilizadores, perfis e roles.
6. JWT ou mecanismo equivalente documentado.
7. Gestão inicial de entidades comuns.
8. Contratos de API estáveis.
9. Tratamento consistente de erros.
10. Logs e `correlationId`.
11. Auditoria das operações relevantes.
12. Documentação OpenAPI/Swagger.
13. Frontend-base responsivo.
14. Página inicial de login.
15. Dashboard após autenticação.
16. Navegação para todos os módulos da plataforma.
17. Páginas de módulos já funcionais.
18. Páginas preparadas para módulos futuros.
19. Dados de demonstração fictícios.
20. Testes unitários, de integração, de API e de aceitação.
21. Docker ou instruções equivalentes de execução.
22. `.env.example` sem segredos reais.

23. README completo.
24. Guia de contribuição.
25. Convenções de branches, commits e Pull Requests.
26. Scripts de instalação, migração, seed, testes e execução.
27. Arquitectura documentada.
28. Critérios de integração para os grupos.

Não entregues apenas ficheiros vazios, pseudocódigo ou ecrãs sem backend. Todas as partes fundamentais devem possuir pelo menos uma implementação funcional mínima ou uma estrutura explicitamente preparada para extensão.

4. Decisões tecnológicas de referência

Usa a seguinte stack, salvo impossibilidade técnica devidamente explicada:

Camada	Tecnologia de referência
Backend	Node.js 22, TypeScript e Express
Validação	Zod ou biblioteca equivalente
Base de dados	PostgreSQL
ORM	Prisma
Autenticação	JWT com access token e refresh token, se implementado
Passwords	Argon2 ou bcrypt com configuração segura
Frontend	React + TypeScript + Vite
Estilos	Tailwind CSS ou CSS modular consistente
Testes backend	Vitest/Jest + Supertest
Testes frontend	Vitest + Testing Library
Documentação API	OpenAPI/Swagger
Execução local	Docker Compose
Controlo de versões	Git
CI	GitHub Actions, se disponível

Se escolheres outra tecnologia, mantém exactamente os mesmos contratos, responsabilidades, critérios de segurança, estrutura modular e comportamentos observáveis.

5. Arquitectura obrigatória

Implementa uma arquitectura monolítica modular, adequada ao tempo e à finalidade pedagógica da PTP III. A aplicação deve ser um sistema único, mas com módulos logicamente separados.

Usa as seguintes camadas:

Camada	Responsabilidade
Apresentação	Login, dashboard, navegação, formulários, tabelas e páginas de módulos
API	Rotas, autenticação, autorização, validação, serialização e erros
Aplicação	Casos de uso, serviços e orquestração
Domínio	Entidades, regras de negócio e invariantes
Persistência	Prisma/ORM, repositórios, migrações e consultas
Integração	Serviços externos, MQTT, notificações e futuras dependências
Infraestrutura	Configuração, logs, Docker, execução e observabilidade

Não mistures regras de negócio complexas directamente nos controllers. Não coloques consultas à base de dados em componentes React. Não dupliques a lógica de autenticação nos módulos.

6. Estrutura obrigatória do repositório

Cria uma estrutura semelhante à seguinte:

Plain Text

```
smart-campus/
├── apps/
│   ├── api/
```

```
│   └── web/
├── packages/
│   ├── shared-types/
│   ├── api-client/
│   └── validation/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── docs/
│   ├── architecture.md
│   ├── api-conventions.md
│   ├── authorization-matrix.md
│   ├── integration-guide.md
│   ├── decisions/
│   └── openapi.yaml
├── tests/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/
├── .github/
│   ├── workflows/ci.yml
│   └── pull_request_template.md
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── package.json
└── README.md
```

Se optares por um único projecto em vez de monorepo, conserva a separação lógica de `api`, `web`, `shared`, `docs`, `tests` e `prisma`.

7. Módulos obrigatórios da plataforma

O frontend deve apresentar todos os módulos seguintes desde a primeira versão:

Módulo	Estado inicial esperado
Dashboard	Funcional com resumo e indicadores fictícios ou reais
Utilizadores e perfis	Funcional no Core
Autenticação	Funcional
Gestão académica	Estrutura funcional ou parcialmente funcional

Departamentos e cursos	Página e rotas preparadas
Disciplinas e planos curriculares	Página e rotas preparadas
Estudantes e docentes	Estrutura funcional inicial
Matrículas e inscrições	Página preparada
Avaliações e notas	Página preparada
Horários e assiduidade	Página preparada
Edifícios e salas	Funcionalidade inicial recomendada
Laboratórios e equipamentos	Página preparada
Ocorrências e manutenção	Funcionalidade inicial recomendada
Dispositivos IoT	Página preparada
Sensores e leituras	Página preparada
Alertas	Página preparada
Notificações	Página preparada
Relatórios	Página preparada
Auditoria	Funcionalidade inicial recomendada
Administração do sistema	Funcional segundo role

Cada módulo deve ter nome, descrição, ícone ou elemento visual simples, rota própria, estado de implementação e indicação do próximo ponto de extensão. Os módulos futuros devem utilizar páginas de estado vazio bem desenhadas, com mensagem pedagógica, por exemplo: “Este módulo está preparado para integração pelo grupo responsável. O contrato inicial encontra-se documentado em docs/integration-guide.md.”

Não uses ecrãs que pareçam completos quando ainda não existe backend. Diferencia visualmente “Disponível”, “Parcial” e “Em desenvolvimento”.

8. Frontend-base obrigatório

A aplicação deve abrir na página de **login**. O utilizador não autenticado não deve ver páginas protegidas.

Depois do login, apresenta um layout persistente com:

- menu lateral ou navegação responsiva;

- nome Smart Campus;
- identificação do utilizador autenticado;
- role actual;
- botão de terminar sessão;
- breadcrumbs ou indicação da página actual;
- área de notificações preparada;
- navegação para todos os módulos;
- estado visual de cada módulo;
- tratamento de carregamento, erro e ausência de dados.

Cria as seguintes rotas frontend:

Plain Text

```
/login
/dashboard
/users
/academic
/departments
/courses
/subjects
/students
/teachers
/enrolments
/assessments
/schedules
/campus
/buildings
/rooms
/laboratories
/equipment
/incidents
/maintenance
/iot
/devices
/sensors
/readings
/alerts
/notifications
/reports
/audit
/admin
```

Implementa um componente comum `ModulePlaceholder` para áreas futuras e um componente comum `ModuleStatusBadge` com os estados `available`, `partial` e `development`.

Exemplo de configuração de módulos:

Plain Text

```
export type ModuleStatus = 'available' | 'partial' | 'development';

export type CampusModule = {
  key: string;
  label: string;
  path: string;
  description: string;
  status: ModuleStatus;
  requiredRoles?: string[];
};

export const campusModules: CampusModule[] = [
  { key: 'dashboard', label: 'Dashboard', path: '/dashboard', description: 'Vis
  { key: 'academic', label: 'Gestão Académica', path: '/academic', description:
  { key: 'students', label: 'Estudantes', path: '/students', description: 'Dado
  { key: 'rooms', label: 'Salas', path: '/rooms', description: 'Salas e disponi
  { key: 'incidents', label: 'Ocorrências', path: '/incidents', description: 'P
  { key: 'iot', label: 'IoT', path: '/iot', description: 'Dispositivos e leitur
  { key: 'reports', label: 'Relatórios', path: '/reports', description: 'Indica
];
```

9. Modelo de dados inicial

Implementa pelo menos as seguintes entidades e relações:

- `User`;
- `Role`;
- `RefreshToken`, se refresh tokens forem usados;
- `Department`;
- `Course`;
- `Subject`;
- `StudentProfile`;
- `TeacherProfile`;
- `Enrolment`;

- `Room`;
- `Building`;
- `Equipment`;
- `Incident`;
- `MaintenanceAction`;
- `Device`;
- `Sensor`;
- `Reading`;
- `Alert`;
- `Notification`;
- `AuditEvent`.

Começa com entidades essenciais e migrações incrementais. Define IDs, timestamps, estados, chaves estrangeiras, índices e restrições de unicidade. Não inventes relações sem as documentar.

Cada entidade deve possuir documentação com finalidade, campos, relações, validações, permissões e endpoints previstos.

10. API REST e contratos

Versiona a API, por exemplo `/api/v1`. Usa nomes consistentes e respostas JSON. Documenta cada endpoint com método, URL, autenticação, role, parâmetros, corpo, resposta de sucesso, erros possíveis e exemplo.

Endpoints mínimos:

Plain Text

```
POST   /api/v1/auth/login
POST   /api/v1/auth/refresh
POST   /api/v1/auth/logout
GET    /api/v1/me
GET    /api/v1/users
POST   /api/v1/users
GET    /api/v1/rooms
POST   /api/v1/rooms
GET    /api/v1/rooms/:id
GET    /api/v1/incidents
POST   /api/v1/incidents
PATCH  /api/v1/incidents/:id/status
```

```
GET    /api/v1/devices
GET    /api/v1/sensors/:id/readings
GET    /api/v1/alerts
GET    /api/v1/audit-events
```

Cria contratos estáveis para que os grupos consigam desenvolver módulos sem alterar o Core de forma arbitrária. Quando houver alteração incompatível, cria nova versão ou documenta estratégia de migração.

Formato de sucesso sugerido:

JSON

```
{
  "data": {},
  "meta": {
    "correlationId": "sc-2026-example"
  }
}
```

Formato de erro obrigatório:

JSON

```
{
  "code": "ROOM_NOT_FOUND",
  "message": "Sala não encontrada",
  "details": [],
  "correlationId": "sc-2026-example"
}
```

Não devolvas stack traces, queries, tokens, passwords, chaves privadas ou dados sensíveis.

11. Autenticação e autorização

Implementa:

1. Login com validação.
2. Hashing seguro de password.
3. Access token de duração limitada.
4. Refresh token revogável, se utilizado.
5. Logout.
6. Middleware de autenticação.

7. Middleware de autorização por role.
8. Autorização por objecto quando necessário.
9. Utilizador activo/inactivo.
10. Auditoria de login, logout, falhas e alterações críticas.

Roles iniciais:

```
Plain Text

STUDENT
TEACHER
TECHNICIAN
COORDINATOR
ADMIN
```

Matriz inicial:

Operação	Estudante	Docente	Técnico	Coordenador	Admin
Consultar informação pública	Sim	Sim	Sim	Sim	Sim
Consultar perfil próprio	Sim	Sim	Sim	Sim	Sim
Criar ocorrência	Sim	Sim	Sim	Sim	Sim
Tratar ocorrência atribuída	Não	Não	Sim	Sim	Sim
Gerir salas	Não	Não	Parcial	Sim	Sim
Gerir utilizadores	Não	Não	Não	Parcial	Sim
Consultar auditoria	Não	Não	Não	Parcial	Sim

A autorização deve ocorrer no backend. Esconder um botão no frontend não é segurança.

12. Segurança e privacidade

Implementa validação de entradas, limites de tamanho, CORS configurável, headers de segurança, mensagens de erro seguras, rate limiting básico quando aplicável, configuração sem segredos, gestão de sessões, protecção contra acesso horizontal e vertical, logs sem dados sensíveis e princípio do menor privilégio.

Usa dados fictícios de estudantes. Não introduzas dados pessoais reais nem credenciais reais. Cria um `.env.example` e documenta todos os valores necessários.

13. Integração fácil dos módulos

Cria um **guia de integração para grupos**, contendo:

1. Como clonar e executar o Core.
2. Como criar uma branch de módulo.
3. Como consumir a API.
4. Como obter token de desenvolvimento.
5. Como usar os tipos partilhados.
6. Como criar entidade, serviço, rota e teste.
7. Como adicionar migração.
8. Como documentar endpoint.
9. Como declarar dependências.
10. Como abrir Pull Request.
11. Como fazer revisão.
12. Como resolver conflitos.
13. Como actualizar o frontend sem quebrar os restantes módulos.

Cada módulo novo deve seguir este modelo:

Plain Text

```
modules/<module-name>/
├── domain/
├── application/
├── infrastructure/
├── http/
├── schemas/
└── tests/
```

Cada módulo deverá expor um contrato claro e não deve duplicar `User`, `Role`, autenticação, logging, erros ou configuração global.

14. Git e colaboração

Inclui no repositório:

- `CONTRIBUTING.md`;
- modelo de Issue;
- modelo de Pull Request;
- `CODEOWNERS`, se adequado;
- convenção de branches;
- convenção de commits;
- definição de pronto;
- instruções para migrações;
- processo para alteração de contratos.

Branches sugeridas:

```
main
feature/rooms-crud
feature/incidents-create
feature/iot-readings
fix/auth-expired-token
test/integration-rooms
docs/api-contract
```

Commits sugeridos:

```
feat: adicionar endpoint de salas
fix: corrigir validação de capacidade
test: adicionar teste de sala inexistente
docs: actualizar contrato de ocorrências
refactor: separar serviço de autorização
```

Nunca desenvolvas directamente em `main`. Um Pull Request deve conter objectivo, escopo, alterações, testes, riscos, migrações, impacto nos consumidores e evidências.

15. Testes obrigatórios

Cria pelo menos:

1. Testes unitários de regras de domínio.
2. Testes de validação.
3. Testes de autenticação.
4. Testes de autorização.
5. Testes de API.
6. Testes de persistência.
7. Testes de integração entre módulos.
8. Testes frontend das rotas principais.
9. Teste end-to-end do fluxo principal.
10. Casos negativos.

O fluxo transversal mínimo deve ser demonstrável:

```
Plain Text

Login → consultar perfil → consultar sala → criar ocorrência → alterar estado →
```

Prepara também o fluxo futuro:

```
Plain Text

Sensor → ESP32 → Wi-Fi → MQTT → serviço IoT → API → base de dados → dashboard →
```

Os testes devem indicar pré-condições, pedido, resultado esperado, permissões, dados alterados e evidência.

16. Frontend e módulos ainda não desenvolvidos

Mesmo que o backend de um módulo ainda não exista, cria:

- rota frontend;
- item de menu;
- título;
- descrição do objectivo;
- estado de implementação;
- layout vazio coerente;

- indicação do grupo responsável;
- contrato futuro previsto;
- botão ou ligação desactivada quando não houver operação real;
- mensagem que não induza o utilizador a pensar que a função já está pronta.

Não cries funcionalidades simuladas como se fossem reais. Dados fictícios só podem ser claramente marcados como demonstração.

17. Dashboard inicial

O dashboard autenticado deve mostrar, no mínimo:

- utilizador actual;
- role;
- estado do Core;
- número de salas;
- ocorrências abertas;
- dispositivos registados;
- alertas recentes;
- módulos disponíveis;
- módulos em desenvolvimento;
- actividade recente ou auditoria resumida;
- ligação para documentação ou ajuda.

Os indicadores devem vir da API quando existirem. Se forem placeholders, identifica-os explicitamente.

18. Configuração e execução

Cria os seguintes ficheiros:

Plain Text

```
.env.example
Dockerfile
docker-compose.yml
README.md
CONTRIBUTING.md
```

```
prisma/schema.prisma
.github/workflows/ci.yml
```

Inclui comandos equivalentes a:

```bash
cp .env.example .env
npm install
npm run db:generate
npm run db:migrate
npm run db:seed
npm run dev
npm test
npm run build
```

O README deve explicar pré-requisitos, configuração, base de dados, migrações, seed, execução do backend, execução do frontend, Docker, Swagger, testes, troubleshooting, roles, credenciais fictícias e integração de módulos.

19. Seed e dados de demonstração

Cria dados fictícios para:

- um administrador;
- um coordenador;
- um docente;
- um técnico;
- um estudante;
- edifícios;
- salas;
- ocorrências;
- dispositivos e sensores de demonstração;
- alertas fictícios claramente identificados.

As credenciais de desenvolvimento devem ser apresentadas apenas no README local ou em documentação de demonstração, nunca como credenciais reais de produção.

20. Critérios de aceitação

Considera a primeira versão aceite somente se:

- o projecto instalar sem passos ocultos;
- o backend arrancar;
- o frontend abrir no login;
- o login funcionar;
- um utilizador não autenticado não aceder ao dashboard;
- as roles forem aplicadas no servidor;
- a base de dados puder ser criada através de migrações;
- o seed produzir dados de demonstração;
- a API possuir documentação;
- existir endpoint de saúde;
- erros possuírem formato comum;
- logs possuírem `correlationId`;
- pelo menos salas e ocorrências possuírem integração funcional;
- todos os módulos aparecerem no frontend;
- módulos incompletos serem identificados como tais;
- os testes principais passarem;
- Docker ou execução local estarem documentados;
- não existirem segredos versionados;
- outro grupo conseguir consumir o Core seguindo o guia;
- existir demonstração do fluxo transversal;
- as decisões técnicas estarem documentadas.

21. Forma de trabalho obrigatória da IA

Não tentes gerar tudo de forma desorganizada num único passo. Trabalha nas seguintes fases:

Fase A — análise

Apresenta a arquitectura proposta, as tecnologias, a estrutura do repositório, o modelo de dados, os módulos, as dependências e os riscos. Aguarda validação se houver ambiguidade crítica.

Fase B — fundação

Cria configuração, backend mínimo, frontend mínimo, base de dados, endpoint de saúde, Docker, `.env.example`, README e CI.

Fase C — Core funcional

Implementa utilizadores, roles, login, JWT, autorização, erros, logs, auditoria, Swagger e migrações.

Fase D — módulos iniciais

Implementa salas e ocorrências como módulos demonstrativos, mantendo a arquitectura extensível.

Fase E — frontend-base

Implementa login, layout autenticado, dashboard e navegação para todos os módulos, incluindo placeholders explícitos.

Fase F — qualidade

Cria testes, executa verificações, corrige problemas, revê segurança, confirma migrações e valida o fluxo transversal.

Fase G — documentação

Actualiza README, guia de integração, contratos, decisões, matriz de roles, exemplos de API, convenções Git e checklist.

Fase H — entrega

Apresenta a árvore final do projecto, comandos de execução, credenciais fictícias de demonstração, endpoints principais, testes executados, limitações conhecidas e próximos pontos de extensão.

22. Restrições importantes

Não uses dados reais de estudantes. Não publiques segredos. Não cries seis aplicações independentes. Não dupliques o Core em cada módulo. Não coloques regras críticas apenas no frontend. Não marques como concluído um módulo que apenas contém ecrãs sem integração. Não alteres contratos sem documentação e testes. Não introduzas dependências desnecessárias. Não escondas limitações: marca claramente o que é funcional, parcial ou futuro.

A solução deve privilegiar clareza pedagógica, integração progressiva, facilidade de execução, manutenção por estudantes, evidência verificável e capacidade de evolução

para Web, Mobile, IoT, API, base de dados, segurança e deployment.

23. Formato da resposta final esperado da IA

No final, entrega:

1. Resumo da arquitectura.
2. Árvore completa de ficheiros.
3. Instruções exactas de instalação.
4. Instruções de configuração.
5. Comandos de base de dados.
6. Credenciais fictícias de demonstração.
7. Rotas da API.
8. Rotas do frontend.
9. Matriz de roles.
10. Exemplos de pedidos e respostas.
11. Testes executados e resultados.
12. Guia para adicionar módulo.
13. Guia de branches e Pull Requests.
14. Limitações actuais.
15. Próximas extensões.
16. Demonstração do fluxo transversal.

A resposta deve incluir código funcional, não apenas descrições. Sempre que uma parte não puder ser implementada, explica a razão, cria a estrutura de extensão e indica exactamente como os estudantes poderão completá-la.

Fim do prompt-mestre

Nota pedagógica para a equipa

Este prompt deve ser utilizado como ponto de partida para gerar a fundação técnica. Depois da geração, a turma deve rever as decisões, executar o sistema, compreender cada camada, dividir o trabalho em features, criar branches, abrir Pull Requests, testar os contratos e integrar os módulos. A IA não substitui a aprendizagem: deve tornar o sistema

inicial mais consistente, permitindo que os estudantes aprendam através da leitura, execução, alteração, teste e integração do código.

Referências técnicas

[1] OpenAPI Specification
[2] Git Documentation
[3] GitHub Flow
[4] OWASP Authentication Cheat Sheet
[5] OWASP Password Storage Cheat Sheet
[6] OWASP Logging Cheat Sheet
[7] Docker Compose Documentation
[8] GitHub Actions Documentation